Web Servers, Proxy & Load Balancing

Nginx

Apache

Kong

Who am I?

Ziyad Tarek

Sr. SRE/DevOps Engineer @ Rabbit Mart

Email: ziyadtarek180@gmail.com · **Phone:** +201120364754

Certifications

- CKA · CKAD · KCSA · KCNA
- AWS Cloud Practitioner · Terraform Associate · GCP Associate Cloud Engineer
- RHCSA · GitHub Foundations

Course Timeline

Session	Content
Session 1	Web Fundamentals, HTTP, Nginx/Apache, Document Roots, Labs 1
Session 2	Load Balancing, Path Routing, Rate Limiting, Caching, Security, API Gateway, Labs 2–4

Session 1

Web Fundamentals & Web Servers

What happens when you type `https://google.com`?

Several layers work together.

```
Browser  →  DNS  →  Internet  →  Web Server  →  App Server  →  Database
```

Components

Component	Role	Examples
Client	Sends requests	Browser, curl, Postman
DNS	Domain → IP	`google.com` → `142.251.39.174`
Web Server	Receives HTTP, returns responses	Nginx, Apache
App Server	Business logic	Node, Django, PHP
Database	Persists data	PostgreSQL, MySQL

HTTP = Hypertext Transfer Protocol

- **Request-response** model
- **Stateless** — each request independent
- **Port 80** (HTTP), **443** (HTTPS)

HTTP Request

```
GET /users HTTP/1.1
Host: example.com
Accept: application/json
Authorization: Bearer eyJhbGc...
```

Part	Purpose
Method	GET, POST, PUT, DELETE
Path	/users
Headers	Auth, content type, cookies

HTTP Methods

Method	Purpose
GET	Retrieve (idempotent)
POST	Create
PUT	Full update
PATCH	Partial update
DELETE	Remove

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json

{"name": "John"}
```

HTTP status codes

Range	Meaning	Examples
2XX	Success	Request completed successfully
	`200` OK	Resource found and returned
	`201` Created	Resource created (e.g. POST)
	`204` No Content	Success, no body returned
3XX	Redirection	Client should follow another URL
	`301` Moved Permanently	Resource moved permanently
	`302` Found	Temporary redirect
	`304` Not Modified	Use cached version
4XX	Client error	Request invalid or unauthorized
	`400` Bad Request	Malformed request
	`401` Unauthorized	Authentication required

URL Breakdown: `https://example.com/users?id=5`

Part	Name
`https`	Protocol
`example.com`	Host
`/users`	Path
`?id=5`	Query string

URI = identifies resource | **URL** = URI with location (how to get it)

URI only: `urn:isbn:978-0-262-03384-8` · `mailto:user@example.com`
URL: `https://example.com/users?id=5` (includes protocol + host + path)

HTTP vs HTTPS

	HTTP	HTTPS
Port	80	443
Encryption	None	TLS/SSL
Protects	Nothing	Passwords, cookies, tokens

Certificates: Let's Encrypt, certbot

Static vs Dynamic

Type	Served by	Examples
Static	Web server	HTML, CSS, JS, images
Dynamic	App server	APIs, dashboards, forms

```
User → Nginx → App Server → Database
```

Apache vs Nginx — Quick Comparison

	Apache	Nginx
How it works	1 process/thread per request	Event-driven (non-blocking)
Config	.htaccess in each folder	One main config file
Choose when	Shared hosting, .htaccess needed	High traffic, reverse proxy

Nginx handles many connections with less memory. Apache is easier to tweak per folder.

Where Are My HTML Files? (Document Root)

The server looks in one folder for your files — that's the document root.

Server	Default folder
Apache (Linux)	`/var/www/html`
Nginx	`/usr/share/nginx/html`

Request `/` → server looks for `index.html`. Request `/about` → looks for `about` or `about/index.html`.

Where Is the Config File?

Edit these when you change ports, domains, or routing:

Server	Config file(s)
Apache	`/etc/apache2/sites-available/`
Nginx	`/etc/nginx/nginx.conf` + `/etc/nginx/conf.d/`

In our labs we mount `nginx.conf` from the project — edit locally, container uses it.

Nginx Config: The Big Picture

```
nginx.conf
├── worker_processes   → how many workers
├── events             → max connections
└── http
    └── server         → one per site (listen, server_name, root, location)
```

Flow: Each `server` block = one website. Each `location` = one path rule.

Nginx Config: Top-Level Pieces

```
worker_processes 1;        # How many Nginx processes run
events {
  worker_connections 1024;  # Max connections per worker
}
```

For demos we use 1 worker; in production often set to number of CPU cores.

Nginx Config: Server Block

```
server {
  listen 80;              # Accept requests on port 80
  server_name localhost;  # Which domain (localhost = any)
  root /var/www/html;     # Where static files live
  ...
}
```

`root` = the folder for HTML, CSS, images. Without it, Nginx won't serve static files.

Nginx Config: Location Blocks

```
location / {
  index index.html;       # / → serve index.html
}
location /api {
  proxy_pass http://backend:3000;   # /api → send to backend app
}
```

`/` = default static. `/api` = proxy to backend. The path decides static vs proxy.

Apache Config: The Big Picture

```
httpd.conf or sites-available/
├── Listen 80
├── DocumentRoot       → default folder for files
└── <VirtualHost>      → one per site (like Nginx server block)
    ├── ServerName
    ├── DocumentRoot
    └── <Directory>    → permissions, AllowOverride (for .htaccess)
```

Apache Config: VirtualHost Example

```
<VirtualHost *:80>
  ServerName example.com
  DocumentRoot /var/www/html

  <Directory /var/www/html>
    AllowOverride All    # ← Lets .htaccess work here
    Require all granted
  </Directory>
</VirtualHost>
```

One VirtualHost per site. `AllowOverride All` means .htaccess in that folder is read.

Nginx vs Apache: Config Comparison

Concept	Nginx	Apache
One site	`server { }`	`<VirtualHost> </VirtualHost>`
File folder	`root /path`	`DocumentRoot /path`
Path rules	`location /path { }`	`location` or `.htaccess`
Per-folder config	❌ None	✅ `.htaccess`

What Is .htaccess?

Apache-only. A file you put *inside* a folder to change behavior for that folder only.

Use	Example
Redirect	HTTP → HTTPS
Auth	Password-protect /admin
Rewrite	Pretty URLs
Error page	Custom 404

Nginx has nothing like this — use `location` blocks in the main config instead.

.htaccess: When to Use

```
✅ Shared hosting (no root access)
✅ Quick tweak without editing main config
❌ High traffic (Apache reads it every request — slow)
```

Simple example: redirect HTTP to HTTPS:

```
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
```

Reverse Proxy

```
Client  →  Nginx  →  Backend Servers
```

Does: routing, SSL termination, load balancing, caching, security

Reverse Proxy Config

```
location /api {
    proxy_pass http://backend:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

Inspecting HTTP: curl

```
curl -v URL          # Verbose (TLS, headers)
curl -I URL          # Headers only
curl -k URL          # Skip SSL cert verification (insecure)
curl -h              # Help / list options
```

Common options: `-X POST` method · `-d '{"key":"val"}'` body · `-H "Header: val"` · `-b "cookie=val"` · `-H "Authorization: Bearer TOKEN"`

Real Example 1: Rabbitmart API

Embedded dashboard API (JWT in URL + cookies):

```
curl 'https://kodekloud.com//api/embed/card/eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyZXNvdXJjZSI6eyJxdWVzdGlvbiI6MTIwfSwicGFyYW1zIjp7ImlkIjo0MH0sImV4cCI6MTc3MjExNjE0MiwiaWF0IjoxNzcyMTE1NTQxfQ.BOH2s-iMJN3wLP4EhKdtYhvqI_fSaJtB7TQRthA1-QE/query/xlsx?parameters=%7B%7D&format_rows=true&pivot_results=false' \
  -H 'accept: application/json' \
  -H 'accept-language: en-US,en;q=0.9' \
  -H 'cache-control: no-cache' \
  -H 'content-type: application/json' \
  -b '_ga=GA1.2.208401538.1751285360; metabase.DEVICE=20d91ff-ec0c-4e44-a86f-2dd13a00fc78' \
  -H 'pragma: no-cache' \
  -H 'priority: u=1, i' \
  -H 'referer: https://report.rabbitmart.com/embed/question/eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJyZXNvdXJjZSI6eyJxdWVzdGlvbiI6MTIwfSwicGFyYW1zIjp7ImlkIjo0MH0sImV4cCI6MTc3MjExNjE0MiwiaWF0IjoxNzcyMTE1NTQxfQ.BOH2s-iMJN3wLP4EhKdtYhvqI_fSaJtB7TQRthA1-QE' \
  -H 'sec-ch-ua: "Not:A-Brand";v="99", "Google Chrome";v="145", "Chromium";v="145"' \
  -H 'sec-ch-ua-mobile: ?0' \
  -H 'sec-ch-ua-platform: "macOS"' \
  -H 'sec-fetch-dest: empty' \
  -H 'sec-fetch-mode: cors' \
  -H 'sec-fetch-site: same-origin' \
  -H 'user-agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36' \
  -H 'x-metabase-client: embedding-iframe' \
  -H 'x-metabase-embedded: true'
```

Response: `Token is expired`

Headers Explained

Header	Purpose
`accept: application/json`	Want JSON
`-b` (cookies)	Auth, analytics
`referer`	Where request came from
`x-metabase-embedded`	Embedding context
JWT in URL	Auth token

Real Example 2: KodeKloud (curl -I)

```
curl -I https://kodekloud.com/
```

Key headers: `cf-cache-status: HIT`, `server: cloudflare`, `x-frame-options: DENY`

KodeKloud Headers

Header	Meaning
`cf-cache-status: HIT`	Served from CDN cache
`age: 58916`	In cache for X seconds
`x-frame-options: DENY`	No iframes (clickjacking)
`strict-transport-security`	Force HTTPS
`x-content-type-options: nosniff`	Prevent MIME sniffing

Lab 1: Basic Web Servers

```
cd webserver-project
docker compose up -d
```

`./web1`, `./web2`, `./web3` → Apache `htdocs/`

```
curl http://localhost:8081
curl http://localhost:8082
curl http://localhost:8083
```

Session 2

Load Balancing, Caching, Security & API Gateway

Load Balancing

```
        Load Balancer
       /      |      \
   Server1  Server2  Server3
```

Benefits: Availability, performance, fault tolerance

Load Balancing Algorithms

Algorithm	Use Case
Round Robin	General purpose
Least Connections	APIs, long-lived
IP Hash	Session persistence
Weighted	Different server capacity

Lab 2: Load Balancer

```
upstream web_cluster {
    server web1:80;
    server web2:80;
    server web3:80;
}
location / { proxy_pass http://web_cluster; }
```

```
cd webserver-project-LB && docker compose up -d
curl http://localhost   # Rotates web1, web2, web3
```

Path-Based Routing

```
/app1 → web1
/app2 → web2
/app3 → web3
```

Lab 3: Path-Based Routing

```
cd webserver-project-Routes
docker compose up -d
```

```
curl http://localhost/app1
curl http://localhost/app2
curl http://localhost/app3
```

`proxy_pass http://web1:80/` — trailing `/` strips path

Rate Limiting

Problem: DDoS, scraping

Solution: Limit requests per IP

```
limit_req_zone $binary_remote_addr zone=req_limit:10m rate=5r/s;
limit_req zone=req_limit burst=3 nodelay;
limit_req_status 429;
```

→ **429 Too Many Requests**

Lab 4: Rate Limiting

```
cd webserver-project-RL
docker compose up -d
```

Trigger: `for i in {1..15}; do curl -s -o /dev/null -w "%{http_code}\n" http://localhost; done`

Expect: mix of 200 and 429

Caching

Problem: Same data, many users → overloaded DB

Solution: Cache at reverse proxy

```
proxy_cache_path /tmp/nginx_cache keys_zone=my_cache:10m;
location /api { proxy_cache my_cache; proxy_pass http://backend; }
```

Benefits: Faster, less backend load

Forward vs Reverse Proxy

	Reverse	Forward
Position	In front of servers	In front of clients
Protects	Servers	Clients
Examples	Nginx, Cloudflare	Corp proxy, VPN

Security Headers

Header	Purpose
`X-Frame-Options: DENY`	No clickjacking
`Content-Security-Policy`	Script/source control
`Strict-Transport-Security`	Force HTTPS
`X-Content-Type-Options: nosniff`	No MIME sniffing

API Gateway

```
Client → API Gateway → User Service
                     → Order Service
                     → Payment Service
```

Does: Auth, routing, rate limiting, logging

Tools: Kong, AWS API Gateway, Traefik

Kong API Gateway

- Nginx + Lua
- JWT, rate limiting, plugins
- Kubernetes, Docker, cloud

Production Architecture

```
Internet → Cloud LB → Reverse Proxy → Microservices → DB + Redis
          (ALB)         (Nginx/Kong)    (K8s)
```

Real-World Scenario: E-Commerce

Nginx LB + 5 app servers + Redis

User → Nginx (cache HIT → instant) | (MISS → app → DB → cache)

Real-World: Embedded Dashboard

iframe + JWT in URL + cookies

→ `referer`, `x-metabase-embedded`, CORS, `frame-ancestors`

Real-World: CDN (KodeKloud)

Cloudflare → HIT = cache | MISS = origin

Headers: `cf-cache-status`, `age`

Real-World: Path Routing (SaaS)

`/tenant-a/*` → app A | `/tenant-b/*` → app B

Same domain, different backends

Real-World: Rate Limiting (API)

100 req/min limit → excess gets 429

Backend stays healthy

Real-World: Corporate Proxy

Employee → Proxy → Internet

Proxy logs, blocks, enforces policy

Internet sees proxy IP, not employee

Lab Summary

Lab	Project	Purpose
1	webserver-project	3 Apache, direct ports
2	webserver-project-LB	Nginx load balancer
3	webserver-project-Routes	Path routing /app1, /app2, /app3
4	webserver-project-RL	Rate limiting (429)

curl Cheat Sheet

Command	Purpose
`curl -v URL`	Verbose
`curl -I URL`	Headers only
`curl -H "X: y" URL`	Add header
`curl -b "c=v" URL`	Cookies
`curl -X POST -d '{}' URL`	POST JSON

Discussion Questions

1. Web server vs application server?
2. Reverse vs forward proxy?
3. Why load balancing?
4. What does caching solve?
5. When do you need an API Gateway?

Thank You

Questions?